# Data Cleaning & Claude Code for Research

## ECON 692 – Applied Economics Seminar

Andrew Hobbs

University of San Francisco

February 26, 2026

# Today's Agenda

| Time | Duration | Activity |
|---|---|---|
| 4:35 | 5 min | Welcome & check-in |
| 4:40 | 10 min | Claude Code setup verification |
| 4:50 | 40 min | Claude Code intro & CLAUDE.md workshop |
| 5:30 | 10 min | Break |
| 5:40 | 30 min | Data cleaning best practices |
| 6:10 | 10 min | Break |
| 6:20 | 45 min | Claude Code for data tasks |
| 7:05 | 35 min | Work sprint: clean your own data |
| 7:40 | 15 min | Share-outs |
| 7:55 | 5 min | Wrap-up & next week |

# Announcement: Next Week's Schedule

**No regular class on Thursday, March 5.** Instead:

1. **Morning session: Thursday, March 5, 10:00am–12:00pm**
   - Replacement class covering Week 6 material (EDA and early figures)
   - Location: **Lone Mountain 244B**

2. **Evening seminar: Thursday, March 5, 6:30–8:30pm at Stanford**
   - **Bay Area Tech Economics Seminar** with Andrey Fradkin (BU / Amazon)
   - Topic: economics of AI – LLM market competition and the "Coasean Singularity"
   - Location: Simonyi Conference Center, 389 Jane Stanford Way

Both are **strongly encouraged**. I won't take attendance for either.

# Check-in

- How is your data looking? Any surprises when you loaded it?
- Has anyone started cleaning or merging datasets?
- Did everyone get the Claude Code install instructions?

## Reminder

No major milestone this week – use this time to invest in your tools and data pipeline. **Empirical Strategy Draft** due **Friday, March 6**.

# Learning Objectives

Last week you identified *what* to estimate and drew your DAG. This week you set up your tools, clean your data, and push toward **initial results**.

By the end of today, you will be able to:

1. Set up and use **Claude Code** as a research assistant in your terminal
2. Write a **CLAUDE.md** file that gives Claude Code context about your project
3. Apply **tidy data principles** and build a reproducible data pipeline
4. Use Claude Code to **clean, model, and visualize** – aiming for initial results

# Claude Code

# What Is Claude Code?

Claude Code is an AI assistant that lives in your terminal.

- Reads your entire project – files, folders, git history
- Writes and edits code for you
- Runs commands and checks output
- Remembers your conventions via `CLAUDE.md`

Think of it as a collaborator who can read your whole project and write code on the spot.

```
$ claude

> Read my dataset and tell me
  what needs cleaning.

Claude: I found 3 issues in
  data/raw/survey.csv:
  1. 847 missing values in income
  2. State codes are inconsistent
     (CA, Calif., California)
  3. 23 duplicate rows
```

# Why Claude Code for Research?

**The research workflow problem:**

- You spend 60–80% of research time on data cleaning, not analysis
- Repetitive tasks: recoding, reshaping, merging, validating
- Context switching: Stack Overflow $\rightarrow$ code $\rightarrow$ run $\rightarrow$ debug $\rightarrow$ repeat

**What Claude Code changes:**

- Describe what you want in plain English
- Claude Code reads your data, writes the code, runs it
- You review the output and iterate
- Every step is in a script – fully reproducible

*This is not about replacing your thinking. It is about automating the tedious parts so you can focus on your research design.*

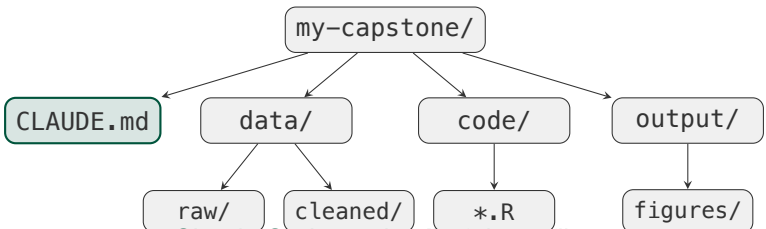
# Setup Verification

```
# Check installation
claude --version

# First launch (log in with your Pro account)
claude

# Test it works
> What directory am I in?
```

**Troubleshooting:** `command not found` → re-run the installer from `docs.anthropic.com` | Login issues → check your email for the Pro invite | Windows → use WSL

**Take 5 minutes now.** Open your terminal and verify Claude Code runs. Raise your hand if you need help.

# How Claude Code Sees Your Project




Claude Code reads **all** of these files
and understands how they relate to each other.

**CLAUDE.md** is the first thing it reads every session.

# CLAUDE.md & Project Setup

# What Is CLAUDE.md?

A **CLAUDE.md** file is a plain-text instruction manual for Claude Code.

It lives in the root of your project and tells Claude Code:

- What your project is about
- How your folders are organized
- What commands to run (R, Python, LaTeX, etc.)
- Conventions and rules to follow
- What data sources you are using

Without CLAUDE.md, Claude Code starts every session from scratch.
With it, Claude Code understands your project **immediately**.

# Anatomy of a Research CLAUDE.md

```
# CLAUDE.MD -- [Your Name] Capstone Project

## Project
- Topic: Effect of [X] on [Y] using [method]
- Language: R (tidyverse, fixest)
- Data: [source], [unit of observation], [time period]

## Folder Structure
- data/raw/     -- original data (never modify)
- data/cleaned/ -- processed data
- code/         -- R scripts (01_clean.R, 02_merge.R, ...)
- output/       -- figures and tables

## Commands
Rscript code/01_clean.R     # cleaning
Rscript code/03_analysis.R  # estimation

## Conventions
- snake_case variable names; NA for missing (never 999)
```

# CLAUDE.md Workshop

**15 minutes.** Create a CLAUDE.md for your capstone project.

1. Open your capstone repository in your terminal
2. Create a file called `CLAUDE.md` in the project root
3. Fill in these sections (use the template from the previous slide):
   - **Project**: one-sentence description, language, data source
   - **Folder structure**: your actual directories
   - **Commands**: how to run your main scripts
   - **Conventions**: any rules you want Claude Code to follow
4. Launch Claude Code in your project: `cd your-repo && claude`
5. Ask: "Read my CLAUDE.md and summarize what you know about my project."

*I will walk around to help. This file will save you hours over the semester.*

# Rules Files

For conventions too detailed for CLAUDE.md, use **rules files** in `.claude/rules/`:

**Example:** `.claude/rules/data-conventions.md`

```
# Data Conventions
- Raw data files are NEVER modified
- All cleaning steps are scripted in code/
- Missing values: NA only (never 999, -1, or "")
- State identifiers: always 2-digit FIPS codes
- Date format: YYYY-MM-DD
```

Claude Code reads rules files automatically every session. They persist your standards across conversations.

*Think of rules files as coding standards that Claude Code enforces for you.*

# Giving Good Instructions

Claude Code is powerful, but **garbage in, garbage out** still applies.

**Vague (avoid):**

- "Clean my data"
- "Fix the problems"
- "Make it work"

**Specific (better):**

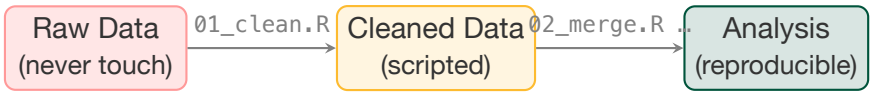
- "Read data/raw/cps.csv. Drop rows where income $< 0$. Recode state names to FIPS codes."
- "Merge enrollment.csv and outcomes.csv on student_id. Report unmatched rows."

**Three principles:**

1. **Be specific** about files, variables, and expected output
2. **State your goal**, not just the steps
3. **Verify**: ask Claude Code to explain what it did, then check

# Data Cleaning Principles

# The Data Pipeline




Every transformation is in a script. Delete `cleaned/` and re-run from scratch.

## Golden Rule of Reproducibility

Raw data is sacred. Scripts are the single source of truth. If you cannot regenerate your cleaned data from raw data + scripts alone, your pipeline is broken.

*Could someone reproduce your current results from raw data + scripts alone? Where does the chain break?*

# Tidy Data Principles

**Tidy data** (Wickham, 2014): every **variable** is a column, every **observation** is a row, every **observational unit** is a table.

**Messy** (wide):

| State | GDP_19 | GDP_20 | GDP_21 |
|---|---|---|---|
| CA | 3.1T | 3.0T | 3.4T |
| TX | 1.8T | 1.7T | 1.9T |

**Tidy** (long):

| State | Year | GDP |
|---|---|---|
| CA | 2019 | 3.1T |
| CA | 2020 | 3.0T |
| CA | 2021 | 3.4T |
| TX | 2019 | 1.8T |

Most panel data methods (DiD, TWFE, event studies) require tidy (long) format.

R: `pivot_longer()` / `pivot_wider()` | Python: `pd.melt()` / `pd.pivot()`

# Common Data Problems

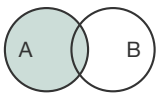
| Problem | Example | R Fix |
|---|---|---|
| Missing values | Income coded as 999, −1, or blank | `mutate(inc = na_if(inc, 999))` |
| Duplicates | Same observation appears twice | `distinct()` or `filter(n() > 1)` |
| Inconsistent coding | “California”, “CA”, “calif.” | `case_when()` or lookup join |
| Type mismatch | Numeric column read as character | `mutate(x = as.numeric(x))` |
| Wide format | Years as column names | `pivot_longer()` |

Python: `df.replace(999, np.nan)`, `df.drop_duplicates()`, `pd.melt()`, `df.astype()`

**Warning:** Claude Code may handle these differently than you expect. Be explicit and check what it did.

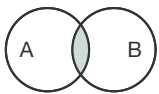
# Merging Datasets

`left_join()`




Keep all of A

`inner_join()`




Keep only matches

**Key decisions:**

- What is the **key**? (state + year? student_id?)
- Are keys **unique**? (many-to-many is almost always a bug)
- What happens to **non-matches**?

**Always check your merge:**

```
# Rows that failed to match
anti_join(A, B, by = "state_fips")
```

Python: `pd.merge(A, B, how="left", indicator=True)`

# Data Validation

After cleaning, **verify your data** before analysis:

1. **Row counts** – expected *n*?
2. **Key uniqueness** – no duplicate (unit, time)?
3. **Range checks** – plausible values?
4. **Missing patterns** – random or systematic?
5. **Balance** – treatment & control in same periods?

```r
# Quick validation in R
nrow(df)
df |> count(state, year) |>
  filter(n > 1)
summary(df$income)
df |> group_by(treated) |>
  summarize(n = n(),
    pct_na = mean(is.na(income)))
```

# File Organization Best Practices

**Recommended structure:**

| | |
|---|---|
| `data/raw/` | Original files |
| `data/cleaned/` | Processed data |
| `code/01_clean.R` | Cleaning |
| `code/02_merge.R` | Merging |
| `code/03_analysis.R` | Estimation |
| `output/` | Figures & tables |
| `paper/` | Written report |

**Why numbered scripts?**

- Clear execution order
- Anyone can reproduce your results by running 01, 02, 03 in sequence
- Claude Code understands the pipeline when it reads your CLAUDE.md

## Reminder

Add `data/raw/` to `.gitignore` if files are large. Keep scripts in Git. Never commit cleaned data – regenerate it from scripts.

# Claude Code for Data Tasks

# Live Demo

**I will now demonstrate a complete data cleaning workflow using Claude Code.**

1. **Load & inspect**: “Read the data and describe what you see.”
2. **Diagnose**: “What needs to be cleaned before I can run my model?”
3. **Clean**: “Fix codes, handle missing values, reshape to long.”
4. **Merge**: “Merge these two datasets. Report unmatched rows.”
5. **Validate**: “Check for duplicates, summarize the panel structure.”
6. **Save**: “Write the cleaned data and commit the script to Git.”

**Key patterns to watch for:**

- Describe and diagnose *before* asking Claude Code to fix
- Always ask for a *script*, not ad hoc code – scripts are reproducible
- Validate after every step – Claude Code catches things you might miss

# Step 1: Load & Inspect

**Before touching anything, understand what you have.**

Ask Claude Code to read your raw data and report:

- How many rows and columns?
- What are the variable names and types?
- How many missing values per column?
- Are there obvious data quality issues?

**Key pattern:** always *describe and diagnose* before asking Claude Code to fix anything. If it does not know what the data looks like, it will guess – and guess wrong.

*This is also your chance to verify Claude Code read the right file and understands its structure.*

# Step 2: Clean & Reshape

**Turn diagnoses into a scripted pipeline.**

Tell Claude Code *exactly* what to do – and ask it to write a script, not run ad hoc commands:

- Which variables need recoding, and *how*?
- How should missing values be handled? (drop? recode? flag?)
- Does the data need reshaping? (wide $\rightarrow$ long for panel methods)
- Where should the output be saved?

**Key pattern:** "Write a script called `code/01_clean.R` that does X, Y, Z and saves to `data/cleaned/`." Scripts are reproducible; ad hoc commands are not.

# Step 3: Merge & Validate

**Combine datasets and immediately check the result.**

When merging, always specify:

- The **key** variables to join on
- The **join type** (left, inner, full) – do not let Claude Code choose
- What to do with **non-matches** – report them with `anti_join()`

After every merge, validate:

- Did the row count change as expected?
- Are there duplicate (unit, time) combinations?
- Are key variables complete across treatment and control groups?

**Key pattern:** Claude Code sometimes catches issues you would miss. Always ask it to validate – but verify its conclusions too.

# Step 4: Save & Commit

**Lock in your work with Git.**

Once your pipeline runs cleanly:

- Commit your **scripts** to Git (not the cleaned data – regenerate it)
- Add `data/cleaned/` to `.gitignore` if files are large
- Update your **CLAUDE.md** to describe the pipeline
- Write a clear commit message: what changed and why

**Test reproducibility:** delete `data/cleaned/`, re-run your scripts, and confirm you get the same result. If you can't, the pipeline is broken.

*You can ask Claude Code to do all of this: "Commit my cleaning scripts, add cleaned data to .gitignore, and update CLAUDE.md with the pipeline description."*

# Hands-On: Clean Your Data with Claude Code

**20 minutes.** Use Claude Code on your own data.

**Setup:**

1. Navigate to your capstone project: `cd ~/path/to/your-repo`
2. Launch Claude Code: `claude`

**Tasks (in order):**

1. Ask Claude Code to read one of your raw data files and describe it
2. Ask it to identify data quality issues
3. Ask it to write a cleaning script (`code/01_clean.R` or `.py`)
4. Run the script and verify the output
5. Ask Claude Code to validate the cleaned data

**If you do not have data yet:** ask Claude Code to help you find and download a public dataset relevant to your topic.

*Raise your hand if you get stuck.*

# Common Claude Code Patterns for Research

| Task | What to say |
|---|---|
| Describe data | “Read data/raw/file.csv and summarize: rows, columns, types, missing values.” |
| Clean a variable | “In 01_clean.R, recode state names to 2-letter abbreviations using a lookup table.” |
| Reshape | “Pivot the wide GDP columns to long format with year and gdp columns.” |
| Merge | “Left join enrollment.csv and outcomes.csv on student_id. Report unmatched rows.” |
| Validate | “Check for duplicate (state, year) pairs and summarize missing values by group.” |
| Plot | “Plot mean employment by treatment group over time, vertical line at policy date.” |

# What Claude Code Gets Wrong

Claude Code is helpful but not infallible. Common failure modes:

1. **Hallucinated variable names** – *Fix:* have it read the data first
2. **Silent data loss** – a merge may drop rows. *Fix:* compare row counts before/after
3. **Wrong join type** – inner when you wanted left. *Fix:* specify explicitly; use `anti_join()`
4. **Outdated syntax** – deprecated functions. *Fix:* run the code and check warnings

**Rule:** always review the code Claude Code writes. You are the researcher; it is the assistant.

*Have you ever had a merge silently drop observations? How would you detect that?*

# Managing Claude Code's Memory

Claude Code has **limited context** – it forgets between sessions and can lose track within a long one.

**How to work around this:**

1. **Commit often with clear messages.** Git is Claude Code's long-term memory – it can read commits and diffs to recover context.
2. **Keep CLAUDE.md current.** Loaded every session; put anything important here.
3. **Use `.claude/rules/`** for conventions you never want to re-explain.
4. **Paste context** when starting fresh: "The script is `code/01_clean.R`. Pick up where we left off."
5. **Tell it to read files** rather than summarize from memory.

**Rule:** if Claude Code seems confused, it lost context. Point it to specific files and commits.

# Work Sprint

# Work Sprint Goals

**35 minutes.** Work on your own project data with Claude Code.

Choose the task that matches where you are:

| If you have... | Do this: |
|---|---|
| Raw data loaded | Clean it: handle missing values, standardize codes, reshape to panel format |
| Multiple datasets | Merge them: identify keys, run joins, validate match rates |
| Clean data already | Validate: run the five checks from the validation slide, then start exploratory plots |
| No data yet | Use Claude Code to help you find and download a relevant public dataset |

**Goal:** by the end of this sprint, you should have a working cleaning script committed to Git.

# Sprint Checkpoint

**Before you stop:**

1. Did you write a **cleaning script** (not just run ad hoc commands)?
2. Can you delete `data/cleaned/` and **regenerate** it from the script?
3. Did you **validate** your cleaned data (row count, no duplicates, plausible values)?
4. Did you **commit and push** to Git?
5. Does your **CLAUDE.md** describe the data pipeline?

If you answered “no” to any of these, take 5 more minutes to fix it.

# Share-outs

**3–4 students, 3 minutes each.**

Cover:

1. **What data problems did you find?**
   - Missing values, inconsistent coding, merge failures, etc.
2. **How did Claude Code help?**
   - What did you ask it to do? Did it work on the first try?
3. **What is your data pipeline now?**
   - Raw → cleaning script → cleaned data → ready for analysis?

Listeners: one tip or one question for each presenter.

# Key Takeaways

1. **CLAUDE.md** is the single most important file for using Claude Code effectively – it gives context that persists across sessions
2. **Raw data is sacred**: every transformation must be in a script, never done by hand
3. **Validate after every step**: row counts, duplicates, merge diagnostics, range checks
4. **Claude Code accelerates the tedious parts** – but you are still responsible for the research design decisions

# Wrap-Up & Next Week

**Due this week:**

- Weekly progress report (Wednesday)

**Next week (Week 6):**

- Exploratory data analysis and early figures
- **Empirical Strategy Draft due Friday, March 6**

**Before next class:**

- Finish your cleaning/merging pipeline and push scripts + CLAUDE.md to GitHub
- Start exploring: summary stats and one or two preliminary figures
- Begin drafting your empirical strategy section (builds on Week 4 DAG work)